Übung 3 – ganz einfach erklärt

Für alle, die noch nie programmiert haben. Du brauchst kein Vorwissen.

So ist dieses Heft aufgebaut

1. **Teil A – Die Grundbegriffe.** Bevor wir die Aufgaben anschauen, lernst du die 10 Bausteine kennen, aus denen *alle* Lösungen bestehen. Jeder Baustein wird mit einem Bild aus dem Alltag erklärt. **Bitte nicht überspringen** – ohne Teil A ist Teil B schwer zu verstehen.
2. **Teil B – Die 6 Aufgaben.** Jede Aufgabe ist gleich aufgebaut:
   - **Worum geht es?** – die Aufgabe in einem Satz
   - **Mit Stift und Papier** – wie du es ohne Computer machen würdest
   - **Der Code, übersetzt** – links Java, rechts was die Zeile auf Deutsch bedeutet
   - **Durchgespielt** – ein Beispiel Schritt für Schritt, wie der Computer es abarbeitet
   - **Wenn dich jemand fragt** – typische Fragen mit einer Antwort, die du so sagen kannst
3. **Teil C – Mini-Wörterbuch.** Zum Nachschlagen, wenn ein Wort unklar ist.

> **Merke:** Der wichtigste Trick beim Verstehen von Code: **Lies ihn laut auf Deutsch vor.** Jede Zeile ist eigentlich ein kurzer Befehl wie „Leg die Zahl 5 in die Schachtel x“. Genau das üben wir hier.

Teil A – Die Grundbegriffe

1. Was ist ein Programm?

> **Stell dir vor …** … ein **Kochrezept**. Es besteht aus Schritten, die man der Reihe nach macht: „Wasser kochen. Nudeln hinein. 10 Minuten warten."

Ein Programm ist genauso eine Liste von Befehlen. Der Computer liest sie **von oben nach unten** und macht **genau** das, was dort steht – nicht mehr und nicht weniger. Er denkt nicht mit.

Jeder Befehl endet in Java mit einem **Strichpunkt** `;` – so wie ein Satz mit einem Punkt endet.

Geschweifte Klammern `{ }` fassen mehrere Befehle zu einer Gruppe zusammen. `{` heißt „hier fängt die Gruppe an", `}` heißt „hier hört sie auf". Die Einrückung (Leerraum am Zeilenanfang) ist nur für uns Menschen, damit man die Gruppen besser sieht.

2. Variable = eine beschriftete Schachtel

> **Stell dir vor …** … eine **Schachtel mit einem Namensschild**. In die Schachtel kann man genau eine Sache hineinlegen. Später kann man nachschauen, was drin ist, oder etwas Neues hineinlegen (dann ist das Alte weg).

```
int alter = 20;
```

Auf Deutsch: *„Mach eine Schachtel für ganze Zahlen, schreib ‚alter' drauf und leg die 20 hinein."*

```
alter = alter + 1;
```

Auf Deutsch: *„Schau nach, was in ‚alter' liegt (20), rechne plus 1 (= 21) und leg das Ergebnis wieder in die Schachtel ‚alter'."*

Achtung: Das `=` ist in Java **kein** „ist gleich" wie in Mathe, sondern heißt **„leg hinein"**. Darum ist `alter = alter + 1` kein Widerspruch. Wenn man wirklich vergleichen will („ist das gleich?"), schreibt man **zwei** Gleichheitszeichen: `==`.

Kurzschreibweisen, die oft vorkommen:

Schreibweise	bedeutet dasselbe wie	auf Deutsch
`x++`	`x = x + 1`	„x um eins erhöhen"
`summe += 5`	`summe = summe + 5`	„5 zur Summe dazuzählen"

3. Datentypen = welche Sorte Schachtel?

Java will bei jeder Schachtel wissen, **was hineinpasst**. Das Wort vor dem Namen sagt es:

Typ	Was passt hinein?	Beispiel
`int`	ganze Zahl (ohne Komma), höchstens ca. **2,1 Milliarden**	`int tage = 31;`
`long`	ganze Zahl, aber eine **viel größere** Schachtel (bis ca. 9 Trillionen)	`long gross = 2147483648L;`

`char`	**ein einzelnes** Zeichen, in einfachen Anführungszeichen	`char buchstabe = 'a';`
`String`	ein Text (beliebig viele Zeichen), in doppelten Anführungszeichen	`String name = "Alice";`
`boolean`	nur **ja** oder **nein** (`true` / `false`)	`boolean gewonnen = false;`

> **Stell dir vor …** … eine **Schuhschachtel** (`int`) und einen **Umzugskarton** (`long`). Wenn etwas zu groß für die Schuhschachtel ist, braucht man den Umzugskarton. Steckt man trotzdem eine zu große Zahl in einen `int`, kommt Unsinn heraus (oft sogar eine negative Zahl). Das nennt man **Überlauf**.

4. Etwas auf dem Bildschirm anzeigen

Befehl	Was passiert
`System.out.println("Hallo");`	schreibt „Hallo“ und springt danach **in die nächste Zeile** (wie die Enter-Taste). *ln* = „line“ (Zeile).
`System.out.print("Hallo");`	schreibt „Hallo“, bleibt aber **in derselben Zeile**.
`System.out.println();`	schreibt nichts, springt nur in die nächste Zeile.
`System.out.printf("%2d ", 5);`	schreibt eine Zahl **in einer bestimmten Form**. `%2d` heißt: „hier kommt eine Zahl hin, mach sie immer 2 Zeichen breit“. Aus 5 wird „ 5“ (mit Leerzeichen davor).

Das `+` zwischen Text und Zahl klebt sie zusammen: `"Versuch " + 3` ergibt den Text „Versuch 3“.

5. if / else = eine Weggabelung

> **Stell dir vor …** … du stehst an einer **Kreuzung**: „**Wenn** es regnet, nimm den Schirm. **Sonst** nimm die Sonnenbrille.“ Du gehst immer nur **einen** Weg.

```
if (zahl == 7) {
    System.out.println("Treffer");
} else {
    System.out.println("Daneben");
}
```

Auf Deutsch: *„**Wenn** die Zahl 7 ist, schreib ‚Treffer‘. **Sonst** schreib ‚Daneben‘.“*

Mit `else if` kann man mehrere Möglichkeiten nacheinander prüfen („Wenn … sonst wenn … sonst …“). Es wird immer nur der **erste** passende Weg genommen.

Zeichen	bedeutet	Zeichen	bedeutet
`==`	ist gleich	`!=`	ist ungleich
`<`	ist kleiner als	`>`	ist größer als
`<=`	kleiner oder gleich	`>=`	größer oder gleich
`&&`	**und** (beides muss stimmen)	`||`	**oder** (eins reicht)

`!`	**nicht** (dreht ja/nein um)

6. Die for-Schleife = etwas mehrmals wiederholen

> **Stell dir vor …** … du läufst **10 Runden** im Stadion und zählst mit: „Runde 1, Runde 2, … Runde 10, fertig.“ Dafür brauchst du drei Dinge: **wo du zu zählen anfängst** (bei 1), **wie lange du läufst** (solange du bei höchstens 10 bist) und **was nach jeder Runde passiert** (eins weiterzählen).

```
for (int runde = 1; runde <= 10; runde++) {
    System.out.println("Runde " + runde);
}
```

Die runde Klammer hat genau diese drei Teile, getrennt durch `;`:

Teil	Code	auf Deutsch
1. Start	`int runde = 1`	„Fang bei Runde 1 an.“
2. Bedingung	`runde <= 10`	„Mach weiter, **solange** die Runde höchstens 10 ist.“
3. Weiterzählen	`runde++`	„Nach jeder Runde eins dazuzählen.“

Alles in den geschweiften Klammern wird bei **jeder Runde** gemacht. Das Programm oben schreibt also „Runde 1“ bis „Runde 10“.

`break;` heißt: *„Hör sofort mit der Schleife auf, auch wenn noch Runden übrig wären.“*

7. Array = ein Eierkarton mit nummerierten Fächern

> **Stell dir vor …** … einen **Eierkarton**. Er hat eine feste Anzahl Fächer, und in jedes Fach passt genau eine Sache. Die Fächer sind durchnummeriert – aber **die Nummerierung beginnt bei 0**, nicht bei 1!

```
int[] zahlen = {1, 2, 3, 4, 5};
```

Fach-Nummer (Index)	0	1	2	3	4
Inhalt	1	2	3	4	5

- `int[]` heißt „ein Karton voller ganzer Zahlen“. Die eckigen Klammern `[]` stehen für „Karton/Array“.
- `zahlen[0]` heißt „was liegt im Fach 0?“ → 1. `zahlen[4]` → 5.
- `zahlen.length` heißt „wie viele Fächer hat der Karton?“ → 5.
- `new long[10]` heißt „mach einen neuen, leeren Karton mit 10 Fächern für große Zahlen“.

> **Merke:** Ein Karton mit 5 Fächern hat die Nummern **0 bis 4**. Darum schreibt man in Schleifen `i < zahlen.length` (kleiner als 5), nicht `<=`.

Typische Schleife, die jedes Fach einmal anschaut:

```
for (int i = 0; i < zahlen.length; i++) {
    // hier etwas mit zahlen[i] machen
}
```

Auf Deutsch: *„Geh mit dem Finger von Fach 0 bis zum letzten Fach. Der Finger heißt i.“*

Alles hinter `//` ist übrigens ein **Kommentar**: eine Notiz für Menschen. Der Computer ignoriert es.

8. Methode = eine kleine Maschine

> **Stell dir vor …** … einen **Toaster**: Du steckst etwas hinein (Brot), die Maschine arbeitet, und es kommt etwas heraus (Toast). Oder einen **Drucker**: Du gibst ihm etwas, er druckt es aus, aber er gibt dir nichts „zurück“.

In der Aufgabe sollst du solche Maschinen bauen. So sieht die erste Zeile (die „Beschriftung“ der Maschine) aus:

```
public static int checkDigit(int[] code)
```

Teil	Bedeutung
`public static`	Steht laut Angabe bei allen Methoden davor. Heißt grob: „Jeder darf diese Maschine benutzen, und man kann sie direkt verwenden.“ Mehr musst du dazu jetzt nicht wissen.
`int`	**Was kommt heraus?** Hier: eine ganze Zahl. Steht hier `void` (= „leer“), kommt **nichts** heraus – die Maschine *macht* nur etwas, z.B. etwas anzeigen (wie der Drucker).
`checkDigit`	Der **Name** der Maschine.
`(int[] code)`	**Was steckt man hinein?** Hier: einen Karton mit Zahlen, der in der Maschine „code“ heißt. Das nennt man **Parameter**.

`return x;` heißt: *„Gib x als Ergebnis heraus und hör auf.“*

Aufrufen = die Maschine benutzen: `checkDigit(meinCode)` heißt „Steck meinCode in die Maschine checkDigit und gib mir das Ergebnis.“

9. Modulo % = der Rest beim Teilen

> **Stell dir vor …** … du teilst **17 Kekse auf 5 Kinder** auf. Jedes Kind bekommt 3 Kekse, und **2 bleiben übrig**. Diese übrig gebliebenen 2 sind der „Rest“.

`17 % 5` ergibt **2**. `110 % 11` ergibt **0** (geht genau auf). Das Zeichen `%` hat hier nichts mit Prozent zu tun.

10. Werkzeuge aus der Werkzeugkiste (die „API“)

Java bringt fertige Werkzeuge mit, die man nicht selbst bauen muss. Damit man sie benutzen darf, muss man sie ganz oben in der Datei „holen“:

```
import java.util.Random;    // Werkzeug für Zufallszahlen holen
import java.util.Scanner;   // Werkzeug zum Lesen von Tastatureingaben holen
```

`new Scanner(System.in)` heißt: *„Bau mir ein neues Lese-Werkzeug, das von der Tastatur liest.“*
Das Wort `new` bedeutet immer „mach ein neues Ding davon“.

Aufgabe 1 – Monatskalender

Worum geht es?

Man bekommt zwei Zahlen: **wie viele Tage** der Monat hat und **an welchem Wochentag** er beginnt (1 = Montag … 7 = Sonntag). Daraus soll ein Kalender wie dieser auf dem Bildschirm erscheinen (31 Tage, Start Mittwoch):

```
      1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31
```

Mit Stift und Papier

Stell dir ein Kästchenpapier mit 7 Spalten vor (Mo bis So). Wie würdest du den Kalender malen?

1. Der Monat beginnt am Mittwoch. Also lässt du **Montag und Dienstag leer** (2 leere Kästchen).
2. Dann schreibst du 1, 2, 3, … in die Kästchen, immer eins weiter nach rechts.
3. Dabei **zählst du mit, in welcher Spalte du bist**. Sobald du in Spalte 7 (Sonntag) geschrieben hast, fängst du eine **neue Zeile** an und bist wieder ganz links.
4. Das machst du, bis du beim letzten Tag (31) bist.

Genau das macht der Code. Die Spalte, in der wir sind, merken wir uns in einer Schachtel namens `column` (englisch für „Spalte“).

Der Code, übersetzt

Java	Auf Deutsch
`public static void oneMonthCalendar(int days, int firstDay) {`	Hier beginnt die Maschine „oneMonthCalendar“. Sie bekommt zwei ganze Zahlen: **days** (Anzahl Tage) und **firstDay** (erster Wochentag). **void**: es kommt nichts heraus, sie zeichnet nur auf den Bildschirm.
`    int column = firstDay - 1;`	Mach eine Schachtel „column“ und leg hinein, **wie viele Kästchen vorne leer bleiben**. Start Mittwoch (3) → 3 − 1 = **2** leere Kästchen (Mo, Di). Start Montag (1) → 0 leere Kästchen.
`    for (int i = 0; i < column; i++) {`	Wiederhole so oft, wie es leere Kästchen gibt (hier 2-mal) …
`        System.out.print("   ");`	… und mal jedes Mal ein leeres Kästchen: **3 Leerzeichen**. Warum 3? Weil jedes Zahlen-Kästchen später auch genau 3 Zeichen breit ist (siehe unten). Nur so stehen die Zahlen genau untereinander.
`    }`	Ende der Wiederholung.
`    for (int day = 1; day <= days; day++) {`	Jetzt zähle mit „day“ von 1 bis zum letzten Tag (z.B. 31). Für **jeden Tag** mach Folgendes:

`System.out.printf("%2d ", day);`	Schreib die Tageszahl, **immer 2 Zeichen breit**, und danach ein Leerzeichen. Aus 5 wird „ 5 “, aus 12 wird „12 “. Jedes Kästchen ist also 3 Zeichen breit.
`column++;`	Wir sind ein Kästchen weiter nach rechts gerückt → Spaltenzähler um 1 erhöhen.
`if (column == 7) {`	**Wenn** wir gerade die 7. Spalte (Sonntag) gefüllt haben …
`System.out.println();`	… dann neue Zeile anfangen (neue Woche) …
`column = 0;`	… und den Spaltenzähler wieder auf 0 setzen, weil wir wieder ganz links sind.
`}`	Ende vom „Wenn“.
`}`	Ende der Wiederholung für alle Tage.
`System.out.println();`	Zum Schluss noch einmal in die nächste Zeile springen. Das verlangt die Angabe („Am Ende wird nochmal eine Leerzeile ausgegeben“).
`}`	Ende der Maschine.

Durchgespielt: `oneMonthCalendar(31, 3)`

Am Anfang: `column = 3 − 1 = 2` → es werden 2 leere Kästchen gemalt. Dann:

day	was geschrieben wird	column danach	column == 7?	was passiert
1	„ 1 “	3	nein	weiter
2	„ 2 “	4	nein	weiter
3	„ 3 “	5	nein	weiter
4	„ 4 “	6	nein	weiter
5	„ 5 “	7	**ja**	**neue Zeile**, column = 0
6	„ 6 “	1	nein	weiter (neue Woche, Montag)
…	…	…	…	…
12	„12 “	7	**ja**	**neue Zeile**, column = 0

Der 5. ist also ein Sonntag – genau wie im Beispielbild.

Wenn dich jemand fragt …

Warum `firstDay - 1`?
Weil man die leeren Kästchen *vor* dem ersten Tag braucht. Beginnt der Monat am 3. Wochentag, sind die 2 Tage davor leer.

Warum 3 Leerzeichen bei den leeren Feldern?
Weil jede Zahl mit `%2d` ausgegeben wird, also 2 Zeichen für die Zahl plus 1 Leerzeichen = 3. Ein leeres Feld muss genauso breit sein, sonst rutschen die Spalten.

Wozu ist `column` da?
Damit das Programm weiß, wann eine Woche voll ist. Nach 7 Kästchen kommt eine neue Zeile.

Warum `print` und nicht `println`?
Weil alle Tage einer Woche in derselben Zeile stehen sollen. In die neue Zeile springen wir nur nach dem Sonntag.

Aufgabe 2 – Zufallszahlen selbst berechnen

Worum geht es?

Ein Computer kann eigentlich nicht „würfeln“. Er kann nur rechnen. Darum benutzt man einen Trick: eine Rechenformel, deren Ergebnisse **wie zufällig aussehen**. Man startet mit einer Zahl (dem **Seed** = „Samen“) und rechnet daraus die nächste Zahl aus, daraus wieder die nächste, usw. Wir sollen so **10 Zahlen** ausrechnen und in einem Karton (Array) zurückgeben.

> **Stell dir vor …** … eine **Kette**: Jedes Glied wird aus dem vorherigen gemacht. Wenn du mit demselben Samen anfängst, kommt **immer genau dieselbe Kette** heraus. Darum heißen die Zahlen „Pseudo“-Zufallszahlen (= „scheinbar“ zufällig).

Die Formel

neue Zahl = (a × alte Zahl + c) Rest beim Teilen durch m

mit a = 1 103 515 245, c = 12 345 und m = 2^{31} = 2 147 483 648.

Mit Stift und Papier (bzw. Taschenrechner)

1. Nimm den Startwert (z.B. 0).
2. Rechne: a × Startwert + c. Teil das Ergebnis durch m und nimm den **Rest**. Das ist Zahl Nr. 1. Schreib sie auf.
3. Nimm Zahl Nr. 1 und rechne damit genauso → Zahl Nr. 2. Aufschreiben.
4. So weiter, bis du 10 Zahlen hast.

Warum `long` und nicht `int`? (Das wird sicher gefragt!)

Erinnere dich: in eine `int`-Schachtel passen Zahlen nur bis ca. **2,1 Milliarden** (genau 2 147 483 647).

- **m** ist 2 147 483 648 – also **um 1 zu groß** für einen `int`.
- **a × alte Zahl** wird noch viel größer: ungefähr 1 Milliarde × 2 Milliarden = 2 Trillionen. Das passt erst recht nicht in einen `int`, aber in einen `long`.

Würde man `int` nehmen, käme durch den Überlauf Unsinn heraus. Darum: **long**.

Der Code, übersetzt

Java	Auf Deutsch
`public static long[] lcg(long seed) {`	Maschine „lcg“. Hinein kommt eine große Zahl: der Startwert **seed**. Heraus kommt **long[]** = ein Karton voller großer Zahlen.
`    final long m = 2147483648L;`	Schachtel „m“ mit 2 147 483 648. **final** = „diese Schachtel ist zugeklebt, der Inhalt darf sich nie ändern“ (eine feste Konstante). Das **L** am Ende sagt Java: „Das ist eine große long-Zahl.“ Ohne L würde Java meckern, weil die Zahl zu groß für einen int ist.
`    final long a = 1103515245L;`	Schachtel „a“ mit dem Faktor aus der Angabe.

`    final long c = 12345L;`	Schachtel „c“ mit dem Wert 12 345.
`    long[] numbers = new long[10];`	Mach einen neuen, leeren Karton „numbers“ mit **10 Fächern** (Fach 0 bis 9). Hier kommen die Ergebnisse hinein.
`    long x = seed;`	Schachtel „x“ für die **aktuelle** Zahl. Am Anfang ist das der Startwert.
`    for (int i = 0; i < numbers.length; i++) {`	Wiederhole für jedes Fach des Kartons (i = 0, 1, … 9 → 10-mal):
`        x = (a * x + c) % m;`	Rechne die Formel mit der alten Zahl aus und leg das Ergebnis **als neue Zahl** in x. (Das `*` ist das Mal-Zeichen, `%` der Rest.)
`        numbers[i] = x;`	Leg die neue Zahl in Fach Nummer i.
`    }`	Ende der Wiederholung.
`    return numbers;`	Gib den vollen Karton als Ergebnis heraus.
`}`	Ende der Maschine.

Durchgespielt mit Startwert 0

Runde (i)	alte Zahl x	Rechnung	neue Zahl x	gespeichert in
0	0	(1103515245 × 0 + 12345) % m = 12345 % m	**12 345**	Fach 0
1	12 345	1103515245 × 12345 + 12345 = 13 622 895 711 870 Rest beim Teilen durch 2 147 483 648	**1 406 932 606**	Fach 1
2	1 406 932 606	gleich weiter …	654 583 775	Fach 2
…	…	…	…	…

In Runde 1 sieht man gut, warum wir long brauchen: Das Zwischenergebnis 13 622 895 711 870 ist über 13 Billionen – viel zu groß für einen int.

In main testen

```
long[] randoms = lcg(0);
for (long r : randoms) {
    System.out.println(r);
}
```

Auf Deutsch: *„Benutze die Maschine lcg mit Startwert 0 und leg den Karton, der herauskommt, in ‚randoms‘. Dann nimm **jede Zahl r aus dem Karton** und schreib sie in eine eigene Zeile.“*

Die Schreibweise `for (long r : randoms)` ist eine Kurzform der Schleife: „für jedes r in randoms“. Man braucht keinen Zähler, wenn man einfach jedes Fach anschauen will.

Wenn dich jemand fragt …

Warum ist das „pseudo“-zufällig?
Weil die Zahlen ausgerechnet werden. Mit demselben Startwert kommt immer dieselbe Folge heraus.

Warum long?
m ist größer als der größte int, und a × x wird noch viel größer. In einen int würde das nicht passen und es käme ein falsches Ergebnis heraus.

Wozu das % m?
Damit die Zahlen nicht immer größer werden. Der Rest beim Teilen durch m liegt immer zwischen 0 und m − 1.

Aufgabe 3 – Zahlen-Ratespiel

Worum geht es?

Der Computer denkt sich eine Zahl zwischen 1 und 100 aus. Der Mensch rät. Nach jedem Tipp sagt der Computer, ob die gesuchte Zahl **kleiner** oder **größer** ist. Man hat **höchstens 10 Versuche**. Wer richtig rät, gewinnt; nach 10 falschen Versuchen hat man verloren.

Wir brauchen **zwei Maschinen**: eine, die sich die Zufallszahl ausdenkt, und eine, die das Spiel spielt.

Mit Stift und Papier

Stell dir vor, du spielst das mit einer Freundin. Du bist der Computer:

1. Du denkst dir eine Zahl aus, z.B. 15.
2. Du sagst: „Versuch 1:“ – sie sagt „20“.
3. Du vergleichst: Ist es genau 15? Nein. Ist 15 kleiner als 20? Ja → „Meine Zahl ist kleiner.“
4. Das wiederholst du. **Zwei Gründe, um aufzuhören:** sie hat richtig geraten, **oder** 10 Versuche sind vorbei.
5. Am Ende: Wenn sie *nicht* gewonnen hat, sagst du „Verloren!“.

Damit du am Ende weißt, ob sie gewonnen hat, machst du dir einen **Merkzettel**: Anfangs steht drauf „noch nicht gewonnen“. Wenn sie richtig rät, schreibst du „gewonnen“ drauf. Im Code ist das die Schachtel `won` (englisch: „gewonnen“), Typ `boolean` (ja/nein).

Der Code, übersetzt – Maschine 1: das Spiel

Java	Auf Deutsch
`public static void guessingGame(int numberToGuess) {`	Maschine „guessingGame“. Hinein kommt die **Zahl, die erraten werden soll**. void: es kommt nichts heraus, die Maschine spielt nur.
`    Scanner scanner = new Scanner(System.in);`	Bau ein neues **Lese-Werkzeug**, das von der Tastatur liest, und nenn es „scanner“.
`    boolean won = false;`	Der **Merkzettel**: „gewonnen?“ – am Anfang **nein** (false).
`    for (int attempt = 1; attempt <= 10; attempt++) {`	Zähle die Versuche („attempt“) von 1 bis 10. Für jeden Versuch:
`        System.out.print("Guess number " + attempt + ": ");`	Schreib z.B. „Guess number 1: “. **print** (ohne ln), damit der Mensch seine Zahl **direkt dahinter** in dieselbe Zeile tippt.
`        int guess = scanner.nextInt();`	**Warte**, bis der Mensch eine Zahl eintippt und Enter drückt. Leg die Zahl in die Schachtel „guess“ (= Tipp).
`        if (guess == numberToGuess) {`	**Wenn** der Tipp genau die gesuchte Zahl ist:
`            System.out.println("You won wisenheimer!");`	„Gewonnen!“ hinschreiben.
`            won = true;`	Auf den Merkzettel schreiben: **ja, gewonnen**.

`break;`	Sofort mit dem Raten aufhören (raus aus der Wiederholung).
`} else if (numberToGuess < guess) {`	**Sonst, wenn** die gesuchte Zahl kleiner ist als der Tipp:
`System.out.println("The number AI picked is lower than your guess.");`	„Meine Zahl ist kleiner als dein Tipp.“
`} else {`	**Sonst** (dann bleibt nur noch: die gesuchte Zahl ist größer):
`System.out.println("The number AI picked is higher than your guess.");`	„Meine Zahl ist größer als dein Tipp.“
`}`	Ende der Weggabelung.
`}`	Ende der Wiederholung. Hier landen wir, wenn gewonnen wurde (break) **oder** 10 Versuche vorbei sind.
`if (!won) {`	**Wenn nicht gewonnen** (das ! heißt „nicht“) …
`System.out.println("You lost! Have you ever heard of divide & conquer?");`	… „Verloren!“ hinschreiben.
`}`	Ende vom „Wenn“.
`}`	Ende der Maschine.

Der Code, übersetzt – Maschine 2: Zufallszahl 1 bis 100

Java	Auf Deutsch
`public static int randomNumberBetweenOneAndHundred() {`	Maschine, in die man **nichts** hineinsteckt (leere Klammern) und aus der eine **ganze Zahl** (int) herauskommt.
`Random random = new Random();`	Hol einen neuen **Würfel** aus der Werkzeugkiste und nenn ihn „random“.
`return random.nextInt(100) + 1;`	Würfle eine Zahl. `nextInt(100)` gibt eine Zahl von **0 bis 99** (die 100 selbst kommt nie!). **+ 1** macht daraus **1 bis 100**. Diese Zahl ist das Ergebnis.
`}`	Ende der Maschine.

Achtung: Ohne das `+ 1` könnte 0 herauskommen, aber nie 100. Das ist ein beliebter Fehler.

In main beides verbinden

```
guessingGame(randomNumberBetweenOneAndHundred());
```

Auf Deutsch: *„Benutze zuerst die Zufalls-Maschine. Die Zahl, die herauskommt, steckst du sofort in die Spiel-Maschine.“* (Von innen nach außen lesen – wie bei Klammern in Mathe.)

Durchgespielt: gesuchte Zahl ist 15

attempt	Tipp	gleich 15?	15 kleiner als Tipp?	Ausgabe	won
1	20	nein	ja	„… lower than your guess.“	false
2	10	nein	nein	„… higher than your guess.“	false
3	15	**ja**	–	„You won wisenheimer!“ → break	**true**

Nach der Schleife: `won` ist true → „nicht gewonnen“ stimmt nicht → „You lost!“ wird **nicht** geschrieben. ✔

Wenn dich jemand fragt …

Warum eine for-Schleife bis 10?
Weil man höchstens 10 Versuche hat. Die Schleife zählt die Versuche mit.

Wofür ist `won`?
Die Schleife kann aus zwei Gründen enden: richtig geraten oder Versuche aufgebraucht. Mit `won` weiß man danach, welcher Grund es war, und schreibt nur bei einer Niederlage „You lost!“.

Was macht `break`?
Es beendet die Schleife sofort, damit man nach dem Gewinnen nicht weiterraten muss.

Was ist „divide & conquer“?
„Teile und herrsche“: Man rät immer die Mitte (erst 50, dann 25 oder 75, …). So halbiert sich der Bereich jedes Mal und man findet jede Zahl in höchstens 7 Versuchen.

Aufgabe 4 – Zwei Arrays tauschen

Worum geht es?

Man bekommt zwei Kartons (Arrays) mit Zahlen. Danach soll im ersten Karton stehen, was vorher im zweiten war, und umgekehrt. Das geht aber nur, wenn beide **gleich viele Fächer** haben. Sonst soll die Maschine `false` („hat nicht geklappt“) zurückgeben, ansonsten `true` („hat geklappt“).

Vorher: Karton 1 = {1, 2, 3, 4, 5}, Karton 2 = {100, 99, 60, 44, 2}. Nachher genau umgekehrt.

Die Kernidee: Tauschen braucht einen Zwischenplatz

> **Stell dir vor …** … du hast ein **Glas mit Apfelsaft** und ein **Glas mit Orangensaft** und willst die Säfte tauschen. Du kannst nicht einfach den Apfelsaft ins Orangensaft-Glas kippen – das ist ja noch voll! Du brauchst ein **drittes, leeres Glas**:
>
> 1. Apfelsaft ins leere Glas schütten (zwischenparken).
> 2. Orangensaft ins jetzt leere Apfel-Glas schütten.
> 3. Apfelsaft aus dem Zwischenglas ins jetzt leere Orangen-Glas schütten.

Im Code heißt das dritte Glas `temp` (kurz für „temporär“ = „vorübergehend“). Das machen wir für **jedes Fach** einzeln: Fach 0 tauschen, Fach 1 tauschen, …

Mit Stift und Papier

1. Zähl die Fächer beider Kartons. Sind es unterschiedlich viele? → „Geht nicht“ (false) und fertig.
2. Sonst: Geh Fach für Fach durch und tausch den Inhalt mit dem Zwischenglas-Trick.
3. Am Ende: „Hat geklappt“ (true).

Der Code, übersetzt

Java	Auf Deutsch
`public static boolean swapArrays(int[] array1, int[] array2) {`	Maschine „swapArrays“. Hinein kommen **zwei Kartons** mit ganzen Zahlen. Heraus kommt **boolean** = ja/nein (hat es geklappt?).
`    if (array1.length != array2.length) {`	**Wenn** die Anzahl der Fächer **ungleich** ist …
`        return false;`	… gib „nein, geht nicht“ heraus und **hör sofort auf**. Der Rest wird nicht mehr gemacht.
`    }`	Ende vom „Wenn“. Wer hier ankommt, weiß: beide sind gleich lang.
`    for (int i = 0; i < array1.length; i++) {`	Geh mit dem Finger i durch alle Fächer (0, 1, 2, …). Für jedes Fach:
`        int temp = array1[i];`	Glas 1 ins **Zwischenglas** schütten: den Inhalt von Fach i aus Karton 1 in „temp“ legen.
`        array1[i] = array2[i];`	Glas 2 in Glas 1 schütten: Fach i von Karton 1 bekommt, was in Karton 2 liegt.

`array2[i] = temp;`	Zwischenglas in Glas 2 schütten: Fach i von Karton 2 bekommt den alten Inhalt von Karton 1.
`}`	Ende der Wiederholung.
`return true;`	Gib „ja, hat geklappt“ heraus.
`}`	Ende der Maschine.

Durchgespielt: das erste Fach (i = 0)

Schritt	array1[0]	array2[0]	temp
vorher	1	100	–
temp = array1[0]	1	100	**1**
array1[0] = array2[0]	**100**	100	1
array2[0] = temp	100	**1**	1

Getauscht! Dasselbe passiert danach mit Fach 1, 2, 3 und 4.

Achtung: Ohne `temp` stünde nach der zweiten Zeile in **beiden** Fächern 100 – die 1 wäre für immer verloren.

Warum tauscht man nicht einfach die ganzen Kartons?

Das ist die schwierigste Frage dieser Übung. Eine einfache Erklärung:

Stell dir vor … … die Maschine bekommt nicht die Kartons selbst, sondern nur **Zettel, auf denen steht, wo die Kartons stehen** (wie eine Adresse: „Karton 1 steht im Regal links“). Genauer: sie bekommt **Kopien** dieser Zettel.

Wenn die Maschine nur ihre **Zettel** vertauscht, ändert das an den echten Kartons im Regal gar nichts – und die Zettel von main sind auch noch dieselben wie vorher.

Wenn die Maschine aber zum Regal geht und den **Inhalt** der Kartons umräumt, dann sieht das jeder, der danach in die Kartons schaut – auch main.

Darum muss man **den Inhalt, Fach für Fach** tauschen.

Wenn dich jemand fragt …

Warum braucht man `temp`?
Wie beim Tauschen von zwei vollen Gläsern braucht man einen Zwischenplatz, sonst wird ein Wert überschrieben und ist weg.

Warum zuerst die Länge prüfen?
Weil man sonst bei einem kürzeren Karton auf ein Fach zugreifen würde, das es nicht gibt – das Programm würde abstürzen. Außerdem verlangt die Angabe dann `false`.

Warum nicht einfach `array1 = array2`?
Die Methode hat nur Kopien der „Adresszettel“. Wenn sie die vertauscht, merkt main nichts davon. Nur wenn man den Inhalt der Arrays ändert, ist die Änderung auch in main sichtbar.

Aufgabe 5 – CamelCase

Worum geht es?

Ein Satz soll in eine Schreibweise ohne Leerzeichen umgewandelt werden, in der **jedes Wort mit einem Großbuchstaben beginnt**. Satzzeichen (Punkt, Komma, Apostroph …) fliegen raus. Großbuchstaben mitten im Wort werden klein.

`"my name isn't Alice"` → `"MyNameIsntAlice"`

Der Name kommt vom Kamel (camel): Die Großbuchstaben stehen heraus wie Höcker.

Mit Stift und Papier

Du gehst den Satz **Buchstabe für Buchstabe** durch und schreibst das Ergebnis auf ein neues Blatt. Dabei merkst du dir nur eine Sache: **„Fängt gerade ein neues Wort an?“**

1. Am Anfang: ja, ein neues Wort fängt an.
2. Siehst du ein **Leerzeichen**? → Nicht abschreiben. Aber merk dir: „Der nächste Buchstabe fängt ein neues Wort an.“
3. Siehst du einen **Buchstaben**?
   - Fängt gerade ein neues Wort an → schreib ihn **groß**. Danach: „kein neues Wort mehr“.
   - Sonst → schreib ihn **klein**.
4. Siehst du **irgendwas anderes** (Punkt, Komma, Apostroph)? → Einfach nicht abschreiben.

Den Merkzettel „Fängt ein neues Wort an?“ nennen wir im Code `newWord` (ja/nein, also `boolean`).

Wie macht man selbst aus „m“ ein „M“?

Die Angabe verbietet fertige Werkzeuge wie `toUpperCase()`. Wir müssen also selbst rechnen. Der Trick: Für den Computer ist **jeder Buchstabe eigentlich eine Zahl**.

Großbuchstabe	A	B	C	…	M	…	Z
Zahl dahinter	65	66	67	…	77	…	90
Kleinbuchstabe	a	b	c	…	m	…	z
Zahl dahinter	97	98	99	…	109	…	122

Die Buchstaben liegen also **der Reihe nach** hintereinander. Rechnung für „m“ → „M“:

1. `'m' - 'a'` = 109 − 97 = **12**. Das heißt: m ist **12 Plätze nach dem a**.
2. `'A' + 12` = 65 + 12 = 77 = **'M'**. Wir gehen beim großen Alphabet genauso 12 Plätze weiter.

Zusammen: `ch - 'a' + 'A'`. Für groß → klein genau umgekehrt: `ch - 'A' + 'a'`.

Aus demselben Grund kann man prüfen, ob etwas ein Kleinbuchstabe ist: `ch >= 'a' && ch <= 'z'` heißt „liegt zwischen a und z“.

> **Merke:** Weil beim Rechnen eine Zahl herauskommt, muss man sie mit `(char)` wieder in ein Zeichen zurückverwandeln.

Der Code, übersetzt

Java	Auf Deutsch
`public static String camelCase(String sentence) {`	Maschine „camelCase“. Hinein kommt ein Text (**sentence** = Satz), heraus kommt ein Text (**String**).
`    char[] chars = sentence.toCharArray();`	Zerschneide den Satz in **einzelne Zeichen** und leg jedes in ein Fach eines Kartons „chars“. Aus "my" wird {'m', 'y'}. (So verlangt es die Angabe.)
`    StringBuilder result = new StringBuilder();`	Nimm ein **neues, leeres Blatt**, auf das wir das Ergebnis schreiben. (Ein StringBuilder ist wie ein Notizblock, an den man immer etwas hinten anhängen kann.)
`    boolean newWord = true;`	Merkzettel „Fängt ein neues Wort an?“ – am Anfang **ja**, denn der erste Buchstabe soll groß werden.
`    for (char ch : chars) {`	Nimm **jedes Zeichen** aus dem Karton, eins nach dem anderen, und nenn es „ch“. Für jedes Zeichen:
`        if (ch == ' ') {`	**Wenn** es ein Leerzeichen ist …
`            newWord = true;`	… nicht abschreiben, aber merken: „Als Nächstes fängt ein neues Wort an.“
`        } else if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {`	**Sonst, wenn** es ein Kleinbuchstabe (zwischen a und z) **oder** ein Großbuchstabe (zwischen A und Z) ist:
`            if (newWord) {`	**Wenn** gerade ein neues Wort anfängt:
`                if (ch >= 'a' && ch <= 'z') {`	**Wenn** der Buchstabe klein ist …
`                    ch = (char) (ch - 'a' + 'A');`	… mach ihn **groß** (mit der Rechnung von oben).
`                }`	(Ist er schon groß, bleibt er so.)
`                newWord = false;`	Merkzettel: „Der Wortanfang ist erledigt.“
`            } else {`	**Sonst** (wir sind mitten im Wort):
`                if (ch >= 'A' && ch <= 'Z') {`	**Wenn** der Buchstabe groß ist …
`                    ch = (char) (ch - 'A' + 'a');`	… mach ihn **klein**.
`                }`	(Ist er schon klein, bleibt er so.)
`            }`	Ende der Unterscheidung Wortanfang / mitten im Wort.
`            result.append(ch);`	Schreib den Buchstaben **hinten auf das Ergebnis-Blatt** (append = anhängen).
`        }`	Ende. Alles andere (Punkt, Komma, Apostroph …) passt in keinen Fall → wird einfach **nicht** abgeschrieben.

`    }`	Ende der Wiederholung über alle Zeichen.
`    return result.toString();`	Mach aus dem Notizblock einen fertigen Text und gib ihn heraus.
`}`	Ende der Maschine.

Durchgespielt: `"my isn't"`

Zeichen	Was ist es?	newWord vorher	Was passiert	Ergebnis bisher
m	Buchstabe	ja	groß machen → M, newWord = nein	M
y	Buchstabe	nein	klein lassen	My
(Leerz.)	Leerzeichen	nein	nicht abschreiben, newWord = ja	My
i	Buchstabe	ja	groß machen → I, newWord = nein	MyI
s	Buchstabe	nein	klein lassen	MyIs
n	Buchstabe	nein	klein lassen	MyIsn
'	anderes Zeichen	nein	ignorieren	MyIsn
t	Buchstabe	nein	klein lassen	MyIsnt

Bei `"AnY"`: A ist Wortanfang und schon groß → bleibt A. n bleibt klein. Y ist mitten im Wort und groß → wird y. Ergebnis „Any“. ✔

Wenn dich jemand fragt …

Wofür ist `newWord`?
Das Programm muss wissen, ob der aktuelle Buchstabe ein Wort beginnt. Nur dann wird er groß. Ein Leerzeichen setzt den Merkzettel auf „ja“, der erste Buchstabe danach wieder auf „nein“.

Wie wandelst du klein in groß um, ohne toUpperCase?
Jeder Buchstabe ist intern eine Zahl, und das Alphabet liegt der Reihe nach. Ich rechne aus, an welcher Stelle im Alphabet der Buchstabe steht (`ch - 'a'`) und gehe im großen Alphabet genauso weit (`+ 'A'`).

Wie werden die Satzzeichen entfernt?
Gar nicht aktiv: Es werden nur Buchstaben abgeschrieben. Alles, was kein Buchstabe und kein Leerzeichen ist, wird einfach übersprungen.

Warum StringBuilder?
Er ist wie ein Notizblock, an den man Zeichen für Zeichen hinten anhängen kann. Das ist praktischer und schneller, als jedes Mal einen ganz neuen Text zu bauen.

Aufgabe 6 – Prüfziffer

Worum geht es?

Eine **Prüfziffer** ist eine zusätzliche Ziffer, die aus den anderen Ziffern ausgerechnet wird. Wenn sich später jemand vertippt, passt die Prüfziffer nicht mehr – und man merkt den Fehler. (So etwas steckt z.B. in IBANs oder Buchnummern.)

Wir bekommen einen Karton mit Ziffern und sollen die Prüfziffer nach einem festen Rezept ausrechnen.

Das Rezept aus der Angabe – mit Stift und Papier

Beispiel: Code **3 9 1 5 8**

Fach (Index)	0	1	2	3	4
Ziffer	3	9	1	5	8
Gewicht = Fach + 2	2	3	4	5	6
Ziffer × Gewicht	6	27	4	25	48

1. Jede Ziffer mit ihrem Gewicht malnehmen (siehe Tabelle).
2. Alles zusammenzählen: 6 + 27 + 4 + 25 + 48 = **110**.
3. Durch 11 teilen, den Rest nehmen: 110 ÷ 11 = 10, Rest **0**.
4. 11 minus Rest: 11 − 0 = **11**.
5. Sonderregel: Kommt 10 heraus → 0. Kommt 11 heraus → 5. Hier 11 → **Prüfziffer 5**.

Warum „Fach + 2“? Die Angabe sagt „Position plus 2“. Die erste Position ist im Karton Fach **0** → Gewicht 2.

Warum die Sonderregel? Eine Prüfziffer muss **eine einzige Ziffer** sein (0 bis 9). 10 und 11 sind zweistellig, darum werden sie ersetzt.

Der Code, übersetzt

Java	**Auf Deutsch**
`public static int checkDigit(int[] code)` `{`	Maschine „checkDigit“. Hinein kommt ein Karton mit Ziffern (**code**), heraus kommt eine ganze Zahl: die Prüfziffer.
`    int sum = 0;`	Schachtel „sum“ für die Summe. Startet bei **0** – wie ein leerer Einkaufskorb, in den wir gleich alles hineinlegen.
`    for (int i = 0; i < code.length; i++)` `{`	Geh mit dem Finger i durch alle Fächer. (Wir brauchen die Fachnummer i, weil das Gewicht davon abhängt.)
`        sum += code[i] * (i + 2);`	Nimm die Ziffer in Fach i, mal Gewicht (i + 2), und **zähl das zur Summe dazu**. (Schritt 1 und 2 des Rezepts.)
`    }`	Ende der Wiederholung. Jetzt liegt in sum die Summe (im Beispiel 110).

`    int rest = sum % 11;`	Rest beim Teilen durch 11 ausrechnen (Schritt 3). Im Beispiel 0.
`    int digit = 11 - rest;`	11 minus Rest (Schritt 4). Im Beispiel 11. „digit“ heißt Ziffer.
`    if (digit == 10) {`	**Wenn** 10 herauskommt …
`        digit = 0;`	… stattdessen 0 nehmen (Sonderregel).
`    } else if (digit == 11) {`	**Sonst, wenn** 11 herauskommt …
`        digit = 5;`	… stattdessen 5 nehmen (Sonderregel).
`    }`	Ende der Sonderregeln. In allen anderen Fällen bleibt die Zahl, wie sie ist.
`    return digit;`	Gib die Prüfziffer heraus.
`}`	Ende der Maschine.

Durchgespielt: wie sich `sum` füllt

i	code[i]	Gewicht i + 2	Produkt	sum danach
0	3	2	6	6
1	9	3	27	33
2	1	4	4	37
3	5	5	25	62
4	8	6	48	**110**

Dann: rest = 110 % 11 = 0 → digit = 11 → Sonderregel → **5**. ✔

Wenn dich jemand fragt …

Warum `i + 2`?
Laut Angabe ist das Gewicht „Position plus 2“. Die Fächer im Array beginnen bei 0, also hat die erste Ziffer das Gewicht 0 + 2 = 2.

Warum startet `sum` bei 0?
Weil wir alles dazuzählen. Würde sie bei einer anderen Zahl starten, wäre die Summe falsch.

Wozu die verschiedenen Gewichte?
Damit man auch merkt, wenn zwei Ziffern vertauscht wurden (z.B. 39 statt 93). Ohne Gewichte wäre die Summe gleich und der Fehler fiele nicht auf.

Teil C – Mini-Wörterbuch

Wort	Einfach gesagt
Variable	Eine beschriftete Schachtel, in der ein Wert liegt.
Datentyp	Die Sorte der Schachtel: `int`, `long`, `char`, `String`, `boolean`.
int / long	Ganze Zahl; long ist die größere Schachtel.
char	Ein einzelnes Zeichen, z.B. `'a'`. Intern eine Zahl.
String	Ein Text, z.B. `"Hallo"`.
boolean	Ja (`true`) oder nein (`false`). Gut als „Merkzettel“.
Array	Ein Karton mit nummerierten Fächern. Die Nummerierung beginnt bei 0.
Index	Die Nummer eines Fachs im Array.
.length	Wie viele Fächer ein Array hat.
Methode	Eine kleine Maschine mit Namen: Etwas geht hinein, sie arbeitet, evtl. kommt etwas heraus.
Parameter	Das, was man in die Maschine hineinsteckt (steht in den runden Klammern).
Rückgabewert / return	Das, was aus der Maschine herauskommt.
void	„Leer“: Die Maschine gibt nichts zurück, sie macht nur etwas.
aufrufen	Eine Maschine benutzen, z.B. `camelCase("hallo welt")`.
main	Die Haupt-Maschine. Hier startet das Programm; von hier aus werden die anderen benutzt.
Schleife (for)	Etwas mehrmals wiederholen, mit einem Zähler.
break	Schleife sofort beenden.
if / else	Wenn … sonst … – eine Weggabelung.
% (Modulo)	Rest beim Teilen. 17 % 5 = 2.
++ / +=	Um 1 erhöhen / etwas dazuzählen.
= / ==	`=` heißt „leg hinein“, `==` heißt „ist es gleich?“.
Überlauf	Eine Zahl ist zu groß für ihre Schachtel → es kommt Unsinn heraus.
final	Diese Schachtel darf nie mehr geändert werden (Konstante).
new	„Mach ein neues Ding davon“ (z.B. neuen Karton, neuen Scanner).
import	Ein fertiges Werkzeug aus Javas Werkzeugkiste holen.
API	Javas Werkzeugkiste mit fertigen Werkzeugen (Scanner, Random, …).
Scanner	Werkzeug, das Eingaben von der Tastatur liest.
Random	Werkzeug, das Zufallszahlen würfelt.
StringBuilder	Ein Notizblock, an den man Text hinten anhängen kann.

Kommentar //	Notiz für Menschen, der Computer ignoriert sie.
; und { }	`;` beendet einen Befehl. `{ }` fassen mehrere Befehle zu einer Gruppe zusammen.

Merke: Wenn du eine Aufgabe erklären sollst, geh immer in dieser Reihenfolge vor: **1.** Was soll herauskommen? **2.** Wie würde ich es von Hand machen? **3.** Welche Zeile im Code macht welchen Schritt? Mit diesen drei Schritten kannst du jede der sechs Aufgaben erklären.